

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Практическая работа 3.4 - 3.5

Выполнил:

Полярус Павел

Группа
К3340

Проверил:
Добряков Д. И.

Санкт-Петербург

2022 г.

Задача

ДЗ 4:

1. Разделить монолитное бэкенд-приложение на микросервисы, придерживаясь концепции database-per-service.
2. Спроектировать микросервисную архитектуру: отразить взаимосвязь между микросервисами, описать способы их взаимодействия.
3. Спроектировать разделение единой БД на обособленные БД.
4. Спроектировать и описать эндпоинты для обеспечения межсервисного взаимодействия в формате спецификации OpenAPI.
5. Описать варианты запросов и ответов, задокументировать возможные ошибки.
6. Сформировать документ с конкретными требованиями и шагами для достижения разделения монолита на микросервисы.

ДЗ 5:

1. Подключить и настроить RabbitMQ.
2. Реализовать межсервисное взаимодействие посредством RabbitMQ.

Ход работы

1. Анализ монолита

Исходное приложение Jobby API - биржа труда, реализованная как монолит. Выделяются следующие функциональные области:

Область	Сущности (таблицы)
Аутентификация	users
Профили	job_seekers, employers
Компании	companies
Вакансии	vacancies, vacancy_skills
Резюме	resumes, work_experiences, educations, resume_skills
Отклики	applications
Справочники	countries, cities, industries, skills, employment_types, degree_types

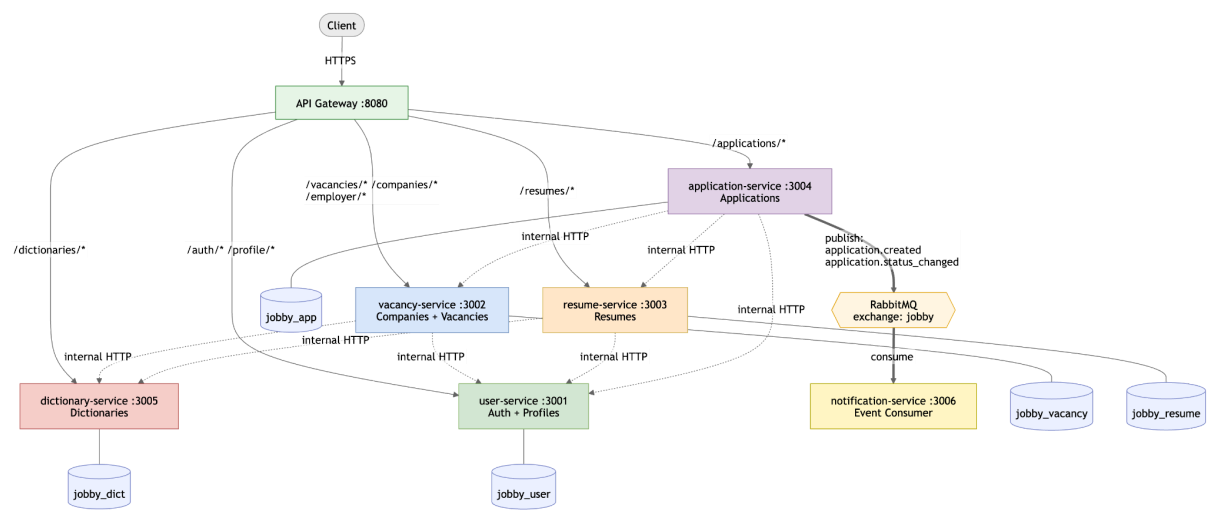
2. Стратегия декомпозиции

Декомпозиция проводилась по принципу Domain-Driven Design (DDD): каждый микросервис соответствует отдельному ограниченному контексту предметной области.

3. Итоговые микросервисы

Сервис	Порт	БД	Таблицы
user-service	3001	jobby_user	users, job_seekers, employers
vacancy-service	3002	jobby_vacancy	companies, vacancies, vacancy_skills
resume-service	3003	jobby_resume	resumes, work_experiences, educations, resume_skills
application-service	3004	jobby_app	applications
dictionary-service	3005	jobby_dict	countries, cities, industries, skills, employment_types, degree_types
notification-service	3006	—	(нет БД, pure consumer)

Схема взаимодействия



4. Разбивка базы данных

User Service

```
-- users
id UUID PK | email VARCHAR UNIQUE NOT NULL | role ENUM(SEEKER,EMPLOYER,ADMIN) NOT NULL
password_hash VARCHAR NOT NULL | phone VARCHAR | created_at | updated_at

-- job_seekers
id UUID PK | user_id UUID UNIQUE FK→users NOT NULL | city_id UUID
first_name VARCHAR NOT NULL | last_name VARCHAR NOT NULL | middle_name VARCHAR
birth_date DATE | gender ENUM(MALE,FEMALE)

-- employers
id UUID PK | user_id UUID UNIQUE FK→users NOT NULL | company_id UUID
first_name VARCHAR NOT NULL | last_name VARCHAR NOT NULL | position VARCHAR
```

Vacancy Service

```
-- companies
id UUID PK | name VARCHAR NOT NULL | description TEXT | website VARCHAR
logo_url VARCHAR | industry_id UUID | city_id UUID | updated_at

-- vacancies
id UUID PK | employer_id UUID NOT NULL | company_id UUID FK→companies NOT NULL
title VARCHAR NOT NULL | description TEXT | conditions TEXT | requirements_note VARCHAR
salary_type ENUM(FIXED,RANGE,FROM,TO,NEGOTIABLE) NOT NULL
salary_fixed INT | salary_min INT | salary_max INT | currency VARCHAR DEFAULT 'RUB'
experience_years_min INT | experience_years_max INT
city_id UUID | industry_id UUID | employment_type_id UUID
is_remote BOOLEAN DEFAULT false | is_published BOOLEAN DEFAULT false | created_at | updated_at

-- vacancy_skills
id UUID PK | vacancy_id UUID FK→vacancies NOT NULL | skill_id UUID NOT NULL
```

Resume Service

```
-- resumes
id UUID PK | job_seeker_id UUID NOT NULL | title VARCHAR NOT NULL | summary TEXT
experience_months_cached INT DEFAULT 0 | is_published BOOLEAN DEFAULT false | created_at | updated_at

-- work_experiences
id UUID PK | resume_id UUID FK→resumes NOT NULL | company_name VARCHAR NOT NULL
role VARCHAR NOT NULL | start_date DATE NOT NULL | end_date DATE | is_current BOOLEAN DEFAULT false

-- educations
id UUID PK | resume_id UUID FK→resumes NOT NULL | degree_type_id UUID NOT NULL
institution VARCHAR NOT NULL | program_name VARCHAR | start_date DATE | end_date DATE

-- resume_skills
id UUID PK | resume_id UUID FK→resumes NOT NULL | skill_id UUID NOT NULL
level ENUM(BEGINNER,INTERMEDIATE,EXPERT) NOT NULL
```

Application Service

```
-- applications
id UUID PK | resume_id UUID NOT NULL | vacancy_id UUID NOT NULL
cover_letter TEXT | status ENUM(PENDING,VIEWED,INVITED,REJECTED,ACCEPTED) DEFAULT PENDING
created_at | updated_at | UNIQUE(resume_id, vacancy_id)
```

Dictionary Service

```
-- countries:      id UUID PK | name VARCHAR UNIQUE NOT NULL
-- cities:         id UUID PK | name VARCHAR NOT NULL | country_id UUID FK-countries NOT NULL
-- industries:     id UUID PK | name VARCHAR UNIQUE NOT NULL
-- skills:         id UUID PK | name VARCHAR UNIQUE NOT NULL
-- employment_types: id UUID PK | name VARCHAR UNIQUE NOT NULL
-- degree_types:   id UUID PK | name VARCHAR UNIQUE NOT NULL
```

5. Межсервисное взаимодействие

5.1 Синхронное (HTTP REST)

Для межсервисных вызовов используются внутренние эндпоинты с префиксом `/internal/`. Они доступны только внутри Docker-сети и защищены заголовком `X-Internal-Secret`.

Вызывающий сервис	Вызываемый сервис	Эндпоинт	Причина
vacancy-service	user-service	GET /internal/employers/by-user/:userId	Проверить, что создающий вакансию пользователь является работодателем
vacancy-service	dictionary-service	GET /internal/cities/:id	Валидация <code>cityId</code> вакансии
vacancy-service	dictionary-service	GET /internal/industries/:id	Валидация <code>industryId</code>
vacancy-service	dictionary-service	GET /internal/skills/:id	Валидация <code>skillId</code> при добавлении навыка
resume-service	user-service	GET /internal/seekers/by-user/:userId	Проверить, что создающий резюме является соискателем
resume-service	dictionary-service	GET /internal/skills/:id	Валидация <code>skillId</code> при добавлении навыка
resume-service	dictionary-service	GET /internal/degree-types/:id	Валидация <code>degreeTypeId</code> при добавлении образования
application-service	resume-service	GET /internal/resumes/:resumeId	Проверить существование резюме и его владельца
application-service	vacancy-service	GET /internal/vacancies/:vacancyId	Проверить существование вакансии
application-service	user-service	GET /internal/employers/by-user/:userId	Проверить, что меняющий статус — автор вакансии

5.2 Асинхронное

Для данного приложения был выбран RabbitMQ по следующим причинам:

- **Простота развёртывания.** Kafka требует отдельного Zookeeper (или KRaft) + брокеров: это как минимум 2 дополнительных контейнера со сложной конфигурацией.
- **Подтверждение доставки.** RabbitMQ имеет встроенный механизм ack/nack на уровне сообщений, что гарантирует обработку уведомления даже при кратковременном сбое consumer.

Топология:

- Тип exchange: topic
- Имя exchange: jobby
- Publisher: application-service
- Consumer: notification-service

Exchange	Routing key	Queue	Publisher	Consumer
jobby	application.created	q.application.created	application-service	notification-service
jobby	application.status_changed	q.application.status_changed	application-service	notification-service

Формат сообщений:

```
application.created:
```

```
{
  "event": "application.created",
  "applicationId": "uuid",
  "vacancyId": "uuid",
  "resumeId": "uuid",
  "seekerUserId": "uuid",
  "createdAt": "2026-06-03T12:00:00.000Z"
}
```

```
application.status_changed:
```

```
{
  "event": "application.status_changed",
  "applicationId": "uuid",
  "newStatus": "INVITED",
  "seekerUserId": "uuid",
  "updatedAt": "2026-06-03T12:00:00.000Z"
}
```

Вывод

В ходе выполнения работы был спроектирован переход от монолитного приложения Jobby к микросервисной архитектуре из 6 сервисов с соблюдением принципа database-per-service.

Синхронное взаимодействие реализуется через HTTP-эндпоинты, защищённые заголовком X-Internal-Secret. Асинхронное взаимодействие реализуется через RabbitMQ с topic exchange jobby: application-service публикует события жизненного цикла отклика, notification-service их потребляет.

Спроектированная архитектура позволяет независимо разрабатывать, тестировать и масштабировать каждый сервис.